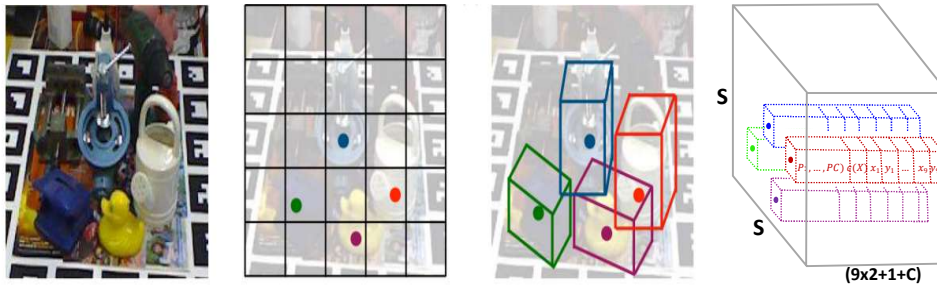


Agenda

- Convolutional Neural Network



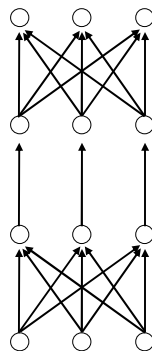
1

Multilayer Perceptron

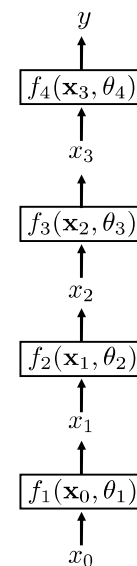
Linear comb of neurons ▷

Neuron-wise nonlinearity ▷

Linear comb of neurons ▷

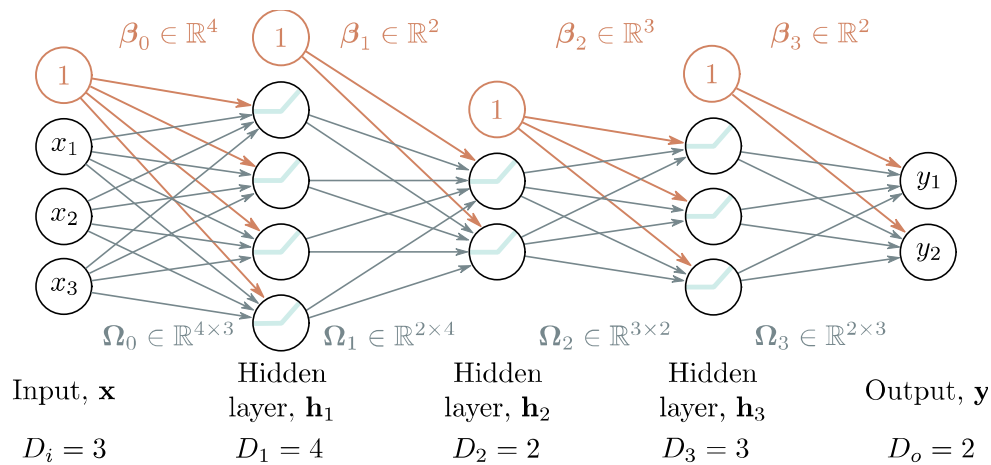


$$[\mathbf{x}, \mathbf{z}_1, \mathbf{h}_1, \dots, \mathbf{z}_{L-1}, \mathbf{h}_{L-1}, \mathbf{y}]$$



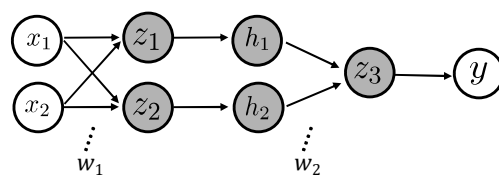
2

Example



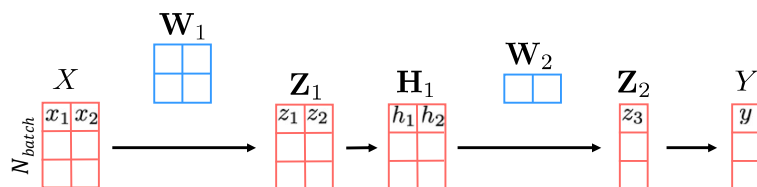
3

Tensor: Multi-dimensional arrays



$$\begin{aligned} \mathbf{z} &= \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 \\ \mathbf{h} &= g(\mathbf{z}) \\ \mathbf{z}_3 &= \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2 \\ y &= 1(z_3 > 0) \end{aligned}$$

Tensor processing with batch size = 3:



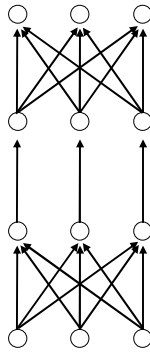
4

Multilayer Perceptron

Linear comb of neurons ▷

Neuron-wise nonlinearity ▷

Linear comb of neurons ▷



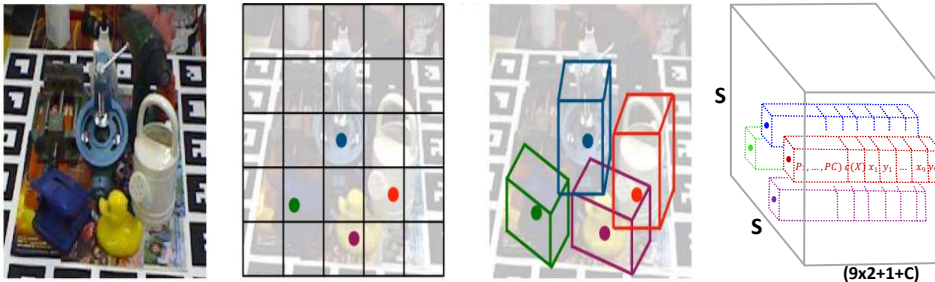
We can pin down “truth” either by adding more data, or by using a more constrained architecture.

- + Universal
- + Simple (elegant theory)
- + Embarrassingly parallel
- Weak inductive biases
- Sample inefficient / data hungry
- Dense (fully-connected) linear layers take a lot of compute

5

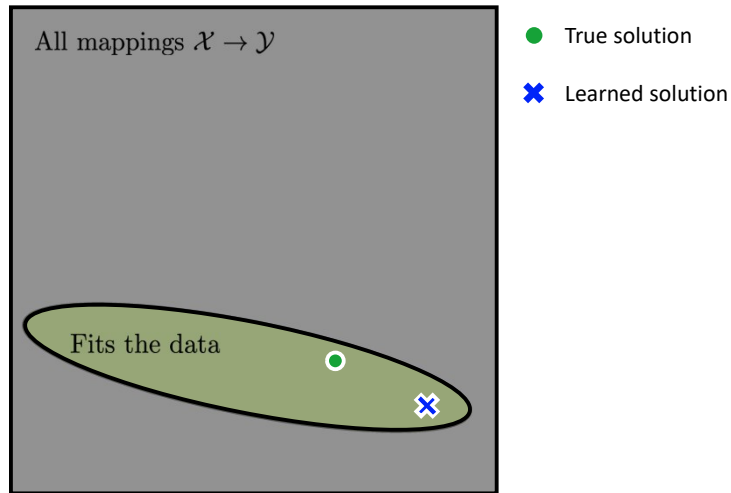
Agenda

- Convolutional Neural Network



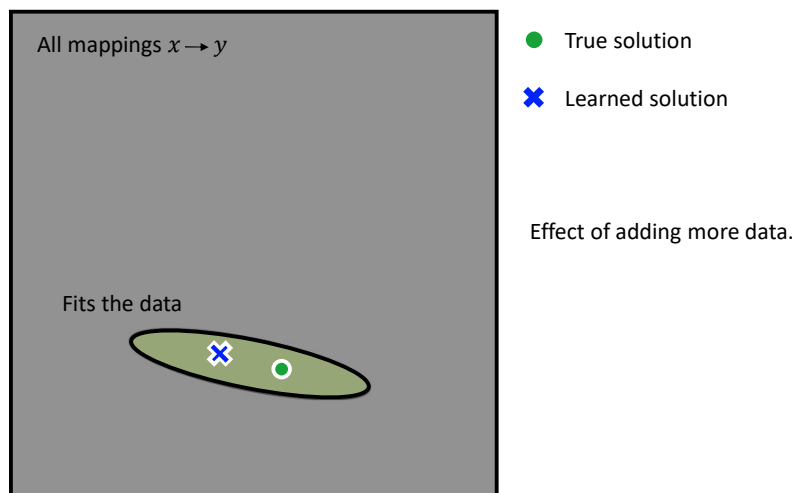
6

Why use other architectures?



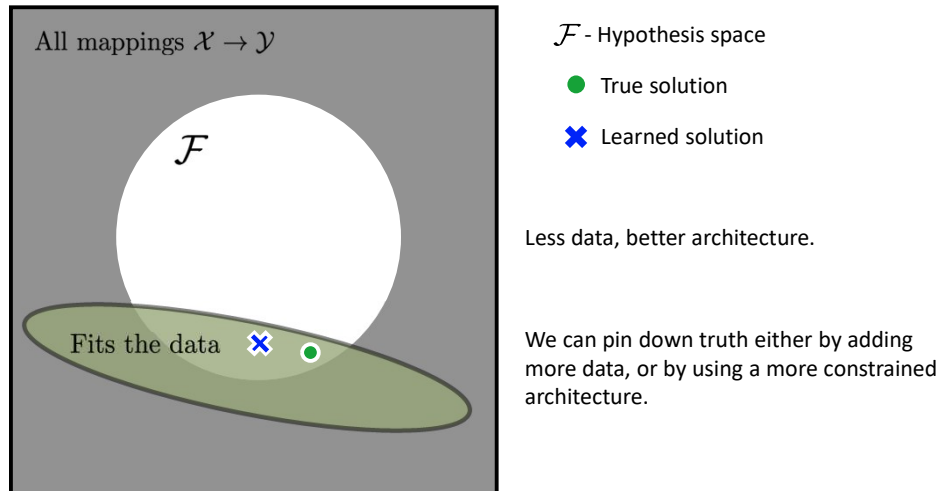
7

Why use other architectures?



8

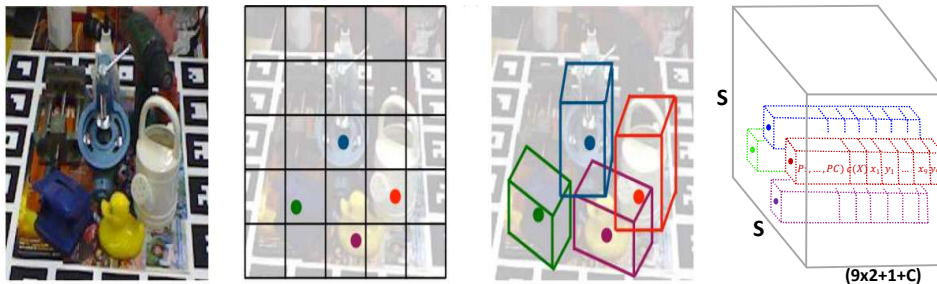
Why use other architectures?



9

Agenda

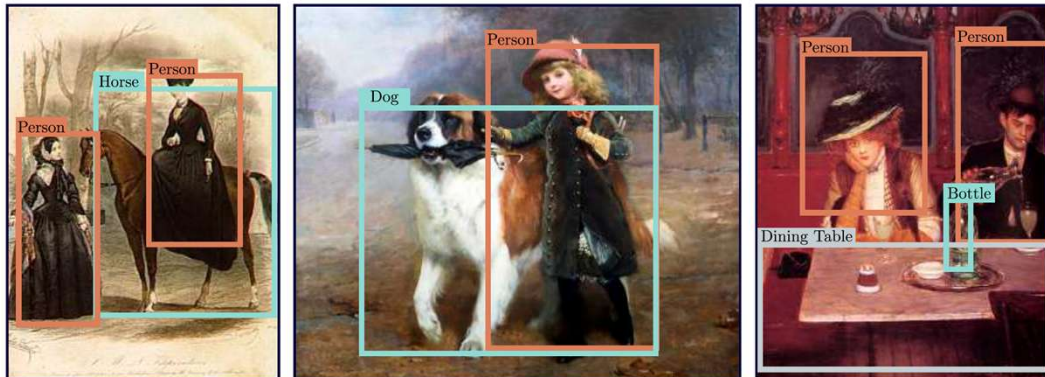
- Convolutional Neural Network



10

Convolutional Neural Network

Many classic computer vision problems are (were?) solved using CNN

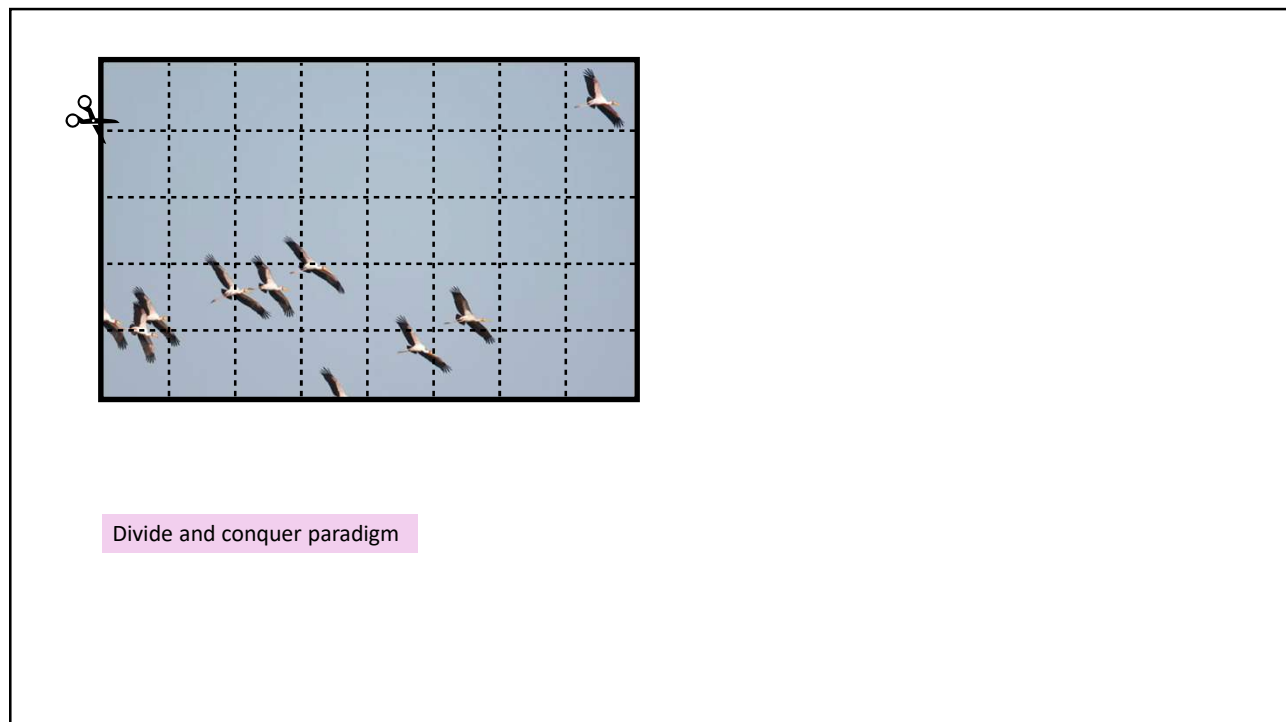


11

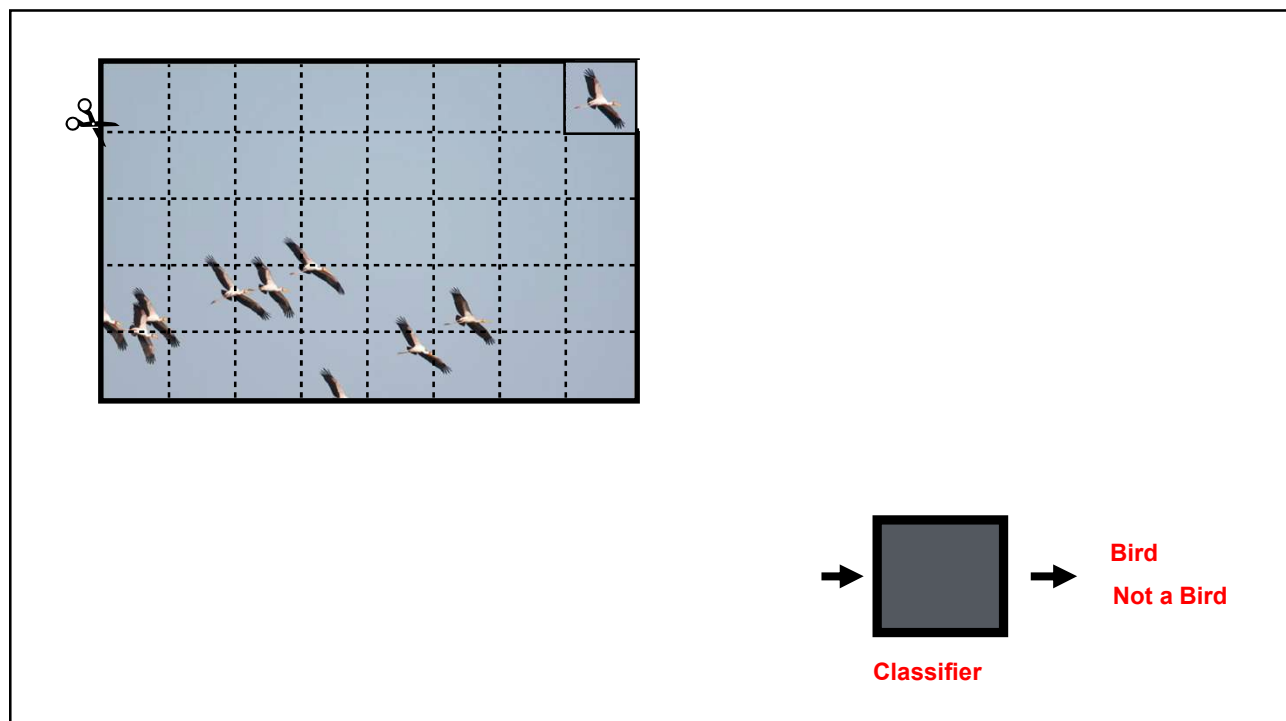
Networks for images

- Problems with fully-connected networks
- Size
 - 224×224 RGB image = 150,528 dimensions
 - Hidden layers generally larger than inputs
 - One hidden layer = $150,520 \times 150,528$ weights = ~22 billion
- Nearby pixels statistically related
- Should be stable under transformations
 - Don't want to re-learn appearance at different parts of image

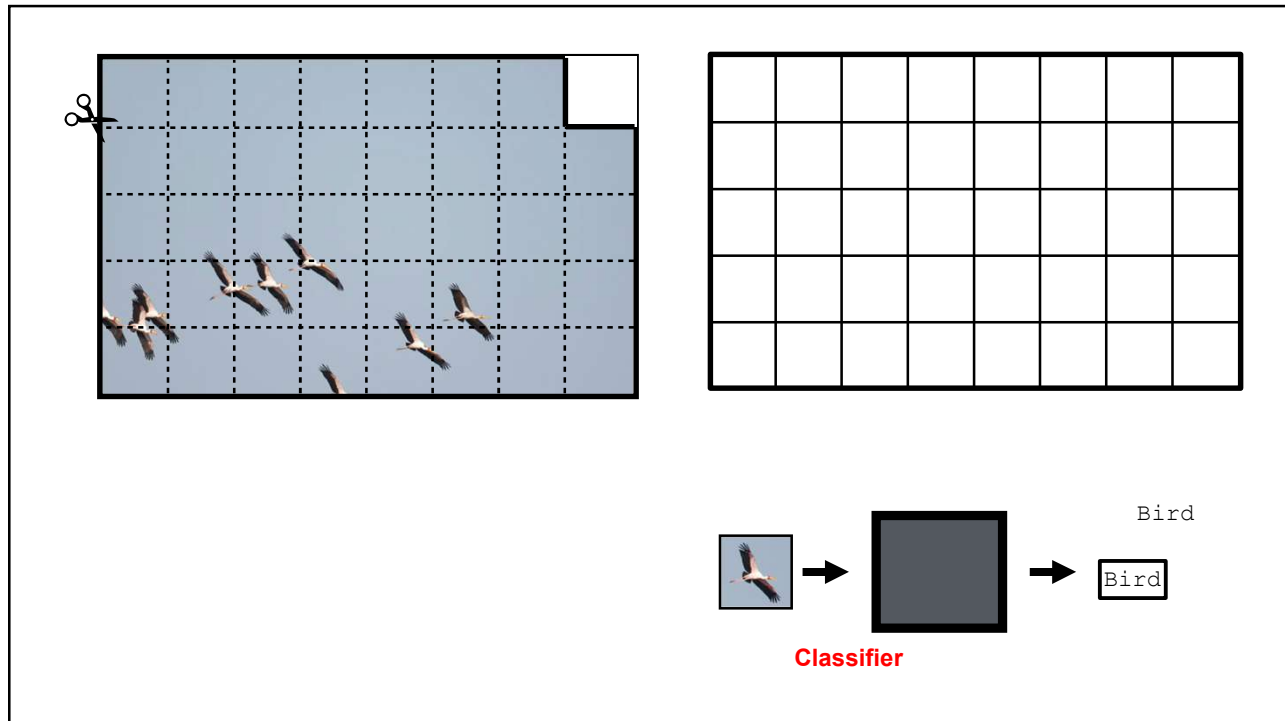
12



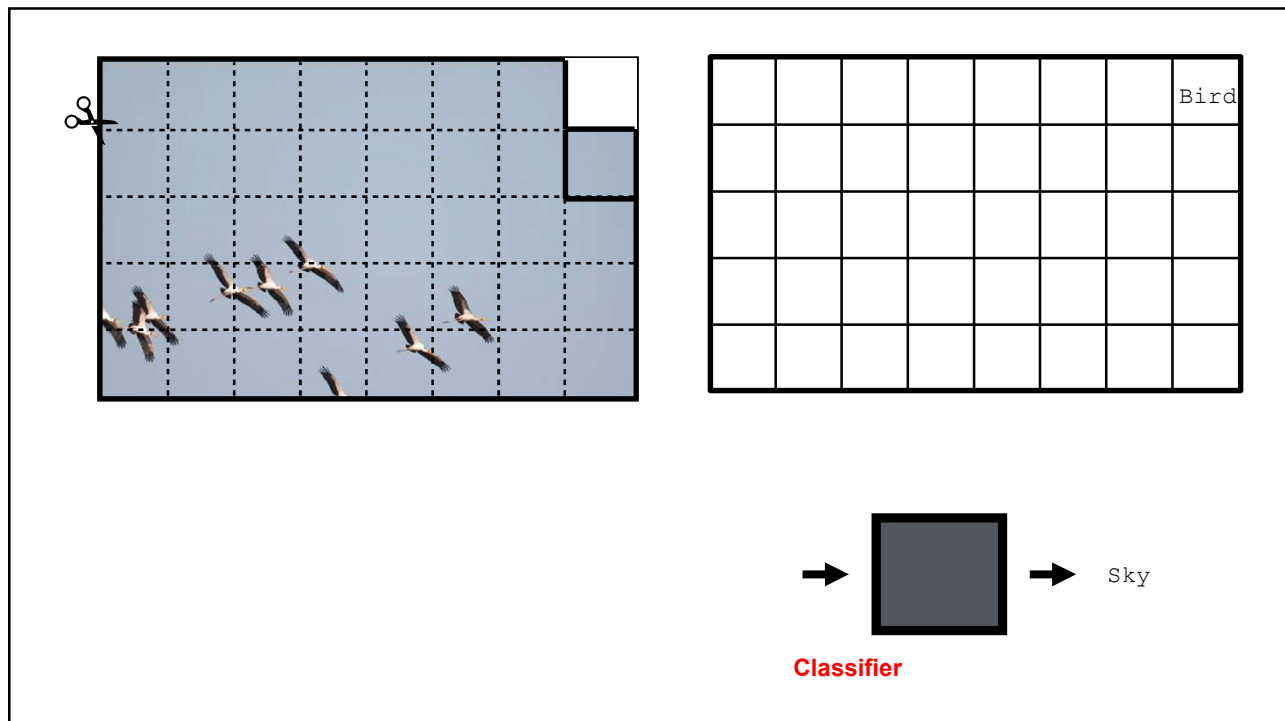
13



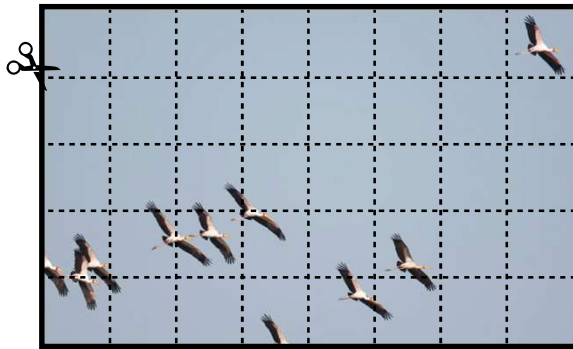
14



15



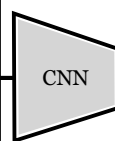
16



Sky	Sky	Sky	Sky	Sky	Sky	Sky	Bird
Sky	Sky	Sky	Sky	Sky	Sky	Sky	Sky
Sky	Sky	Sky	Sky	Sky	Sky	Sky	Sky
Bird	Bird	Bird	Sky	Bird	Sky	Sky	Sky
Sky	Sky	Sky	Bird	Sky	Sky	Sky	Sky

- Parameters see only local image patches
- Parameters are shared across image
- Parallel operation
- Independent and identical

17



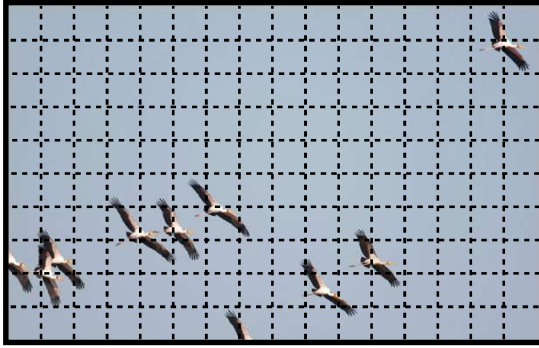
sky	sky	sky	sky	sky	sky	sky	bird
sky	sky	sky	sky	sky	sky	sky	sky
sky	sky	sky	sky	sky	sky	sky	sky
bird	bird	bird	bird	sky	bird	sky	sky
sky	sky	sky	bird	bird	sky	sky	sky

Problem:

What if objects don't fit neatly into these patches?

How to increase the resolution of the output map?

18



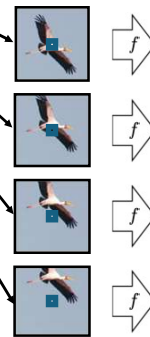
Smaller patches increase resolution, but not easy to recognize content in each small patch

Instead: we use large but overlapping patches

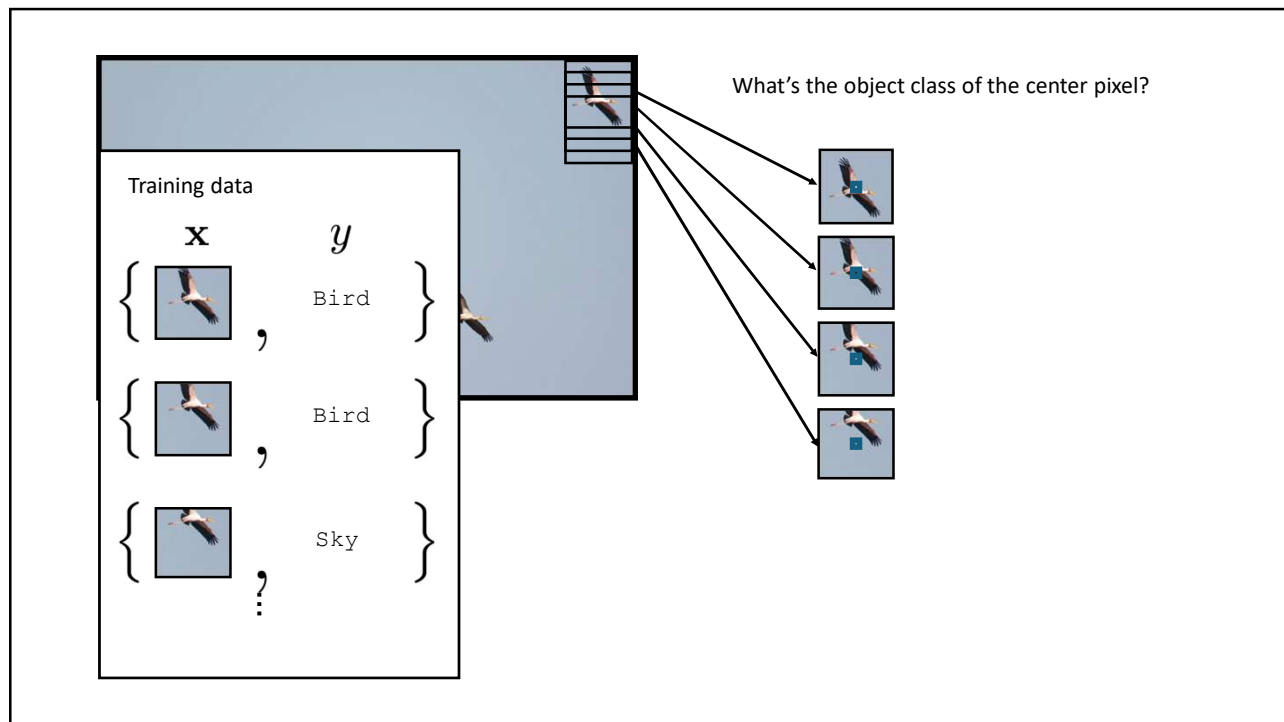
19



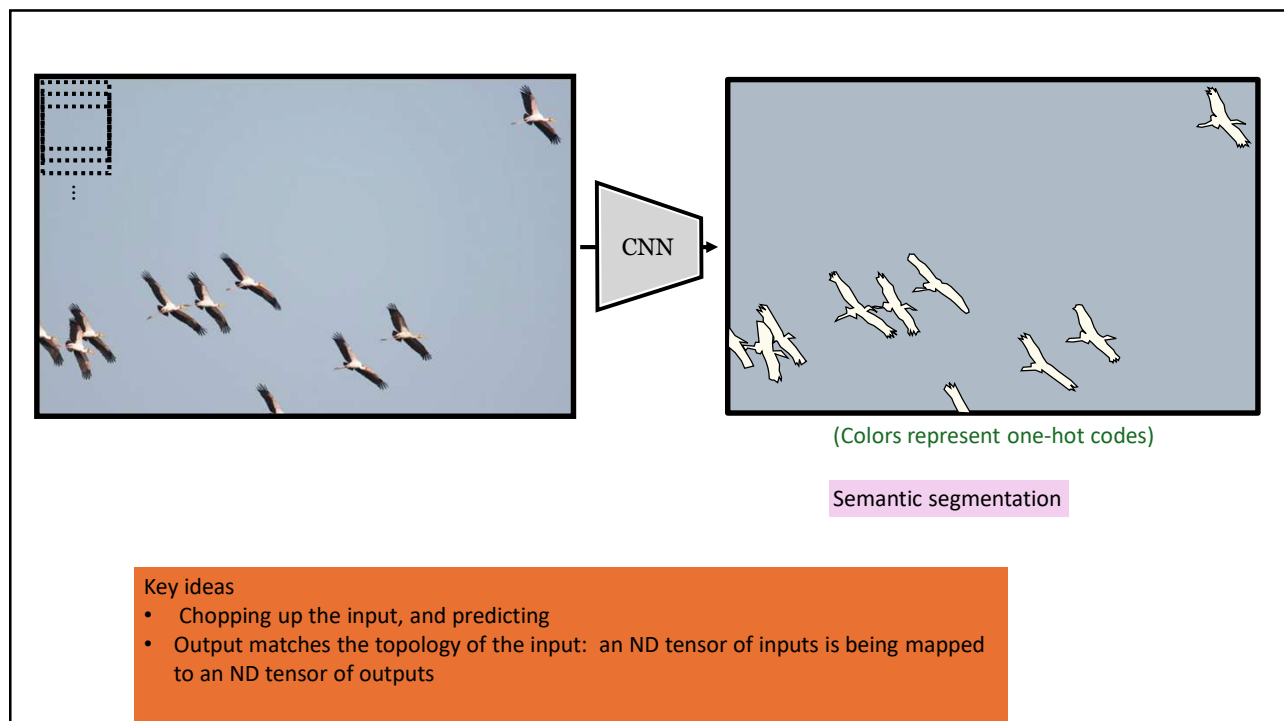
What's the object class of the center pixel?



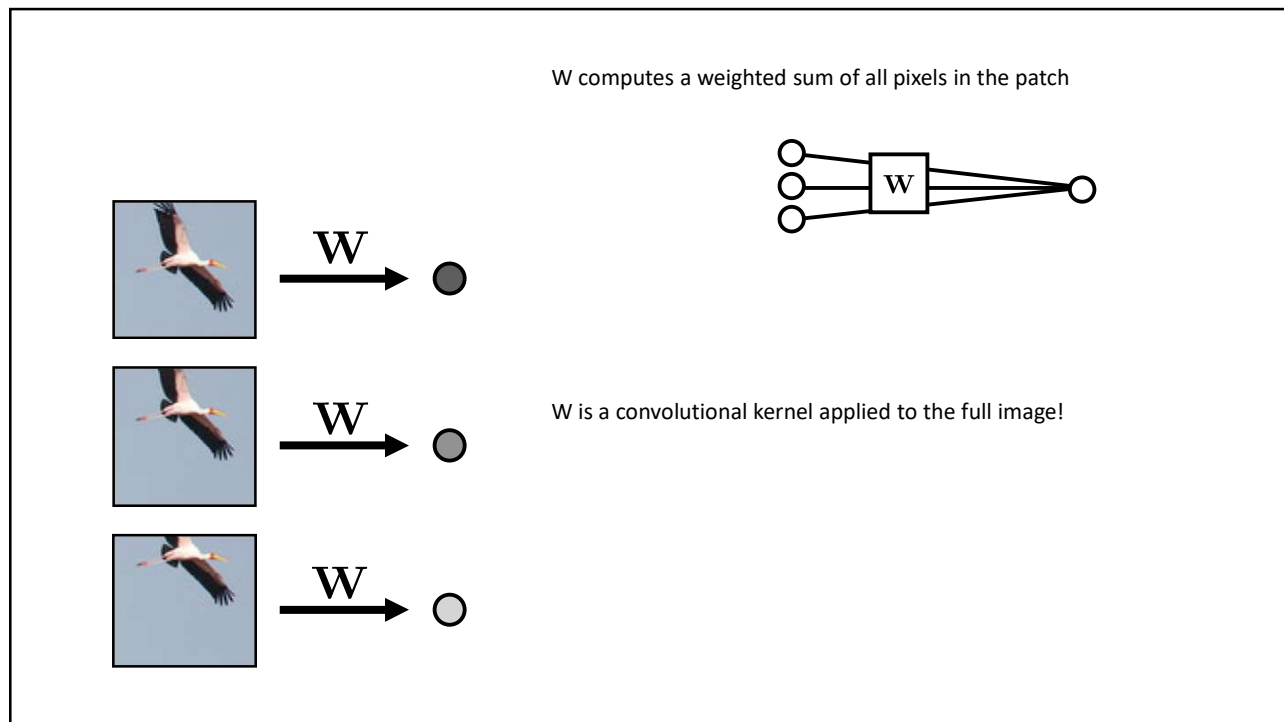
20



21



22



23

Convolution in 1D

- Need to understand the 1D case very well
- Input vector $\mathbf{x}$:

$$\mathbf{x} = [x_1, x_2, \dots, x_I]$$

- Output is weighted sum of neighbors:

$$z_{\underline{i}} = \omega_1 x_{\underline{i-1}} + \omega_2 x_{\underline{i}} + \omega_3 x_{\underline{i+1}}$$

- Convolutional **kernel** or **filter**:

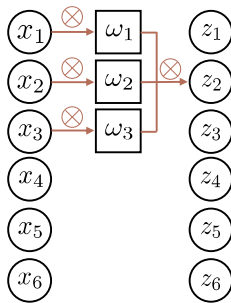
$$\omega = [\omega_1, \omega_2, \omega_3]^T$$

Kernel size = 3

24

Convolution with kernel size 3

a)

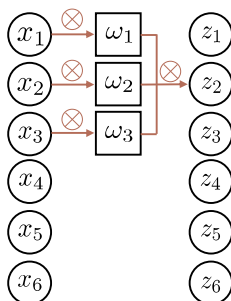


25

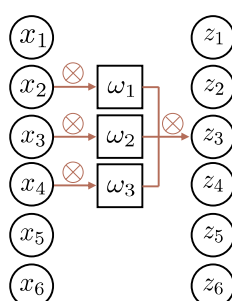
Convolution with kernel size 3

$$z_2 = w_1 x_1 + w_2 x_2 + w_3 x_3$$

a)

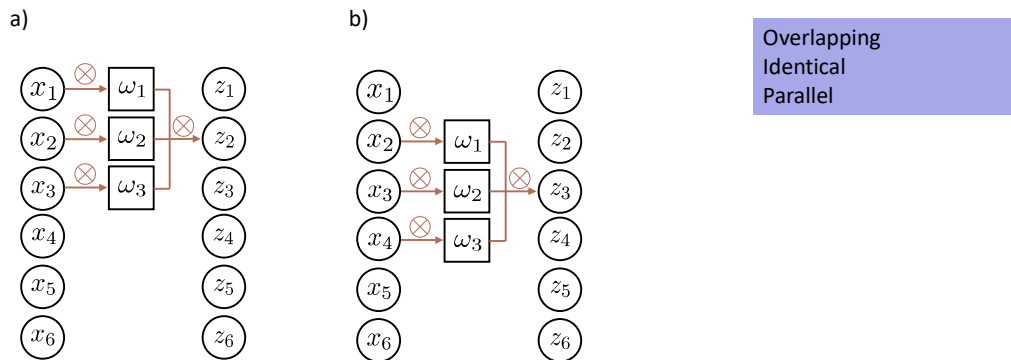


b)



26

Convolution with kernel size 3



27

Special case of fully-connected network

Convolutional network:

$$h_i = a[\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}]$$

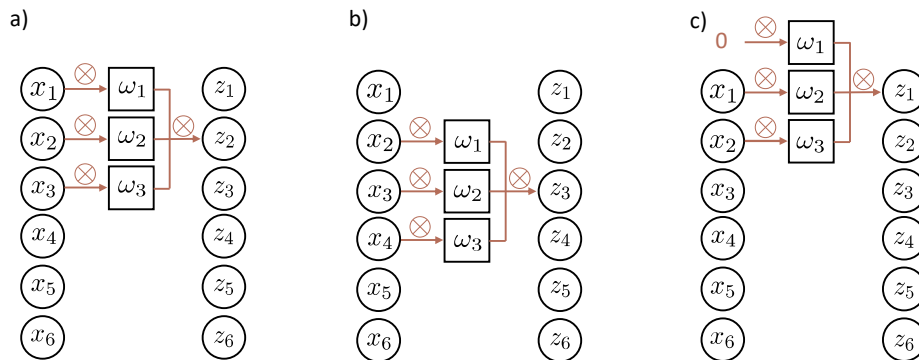
$$= a \left[\beta + \sum_{j=1}^3 \omega_j x_{i+j-2} \right]$$

Fully connected network:

$$h_i = a \left[\beta_i + \sum_{j=1}^D \omega_{ij} x_j \right]$$

29

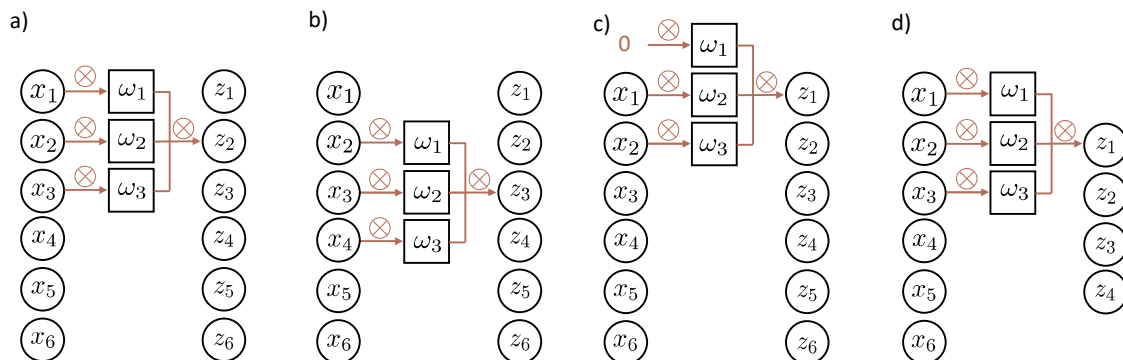
Zero padding



Treat positions that are beyond end of the input as zero.

30

“Valid” convolutions



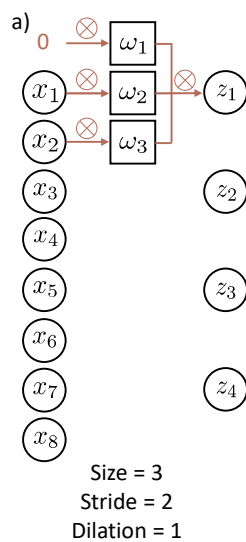
Only process positions where kernel falls in image (smaller output).

31

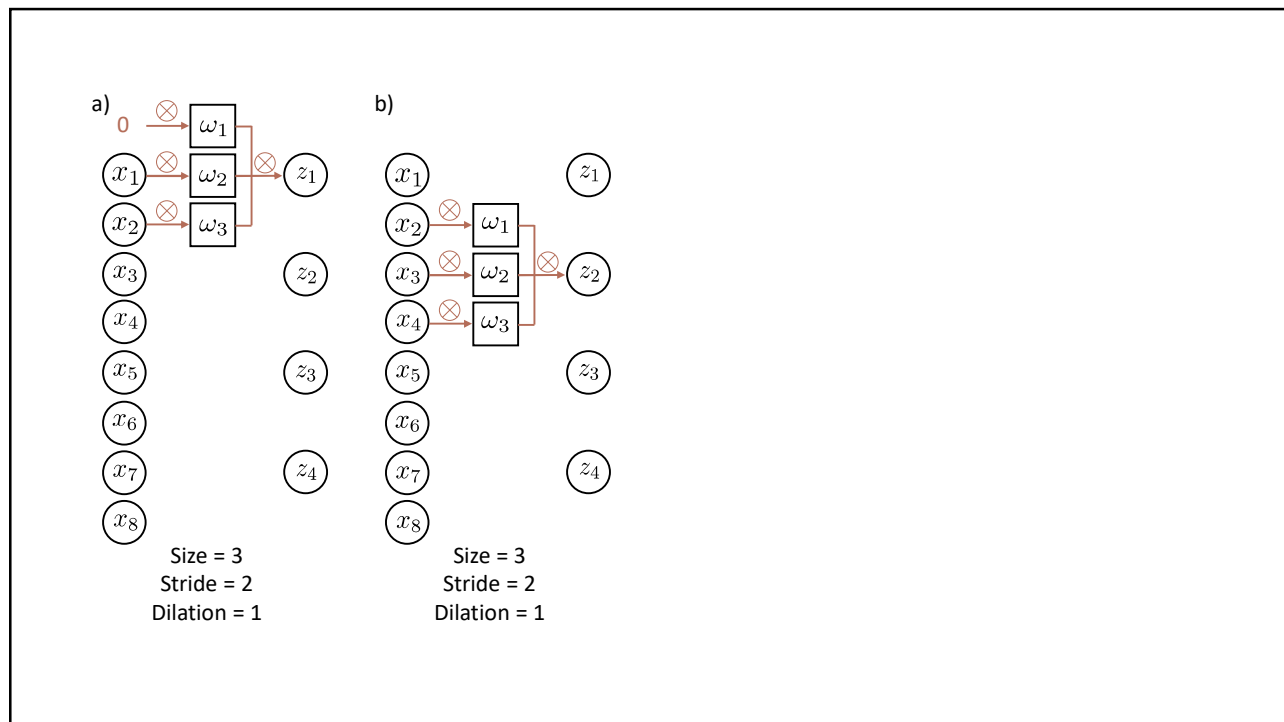
Kernel size, Stride, and dilation

- Kernel size = weight a different number of inputs for each output
 - Combine information from a larger area
 - But kernel size 5 uses 5 parameters
- Stride = shift by k positions for each output
 - Decreases size of output relative to input
- Dilated or atrous convolutions = intersperse kernel values with zeros
 - Combine information from a larger area
 - Fewer parameters

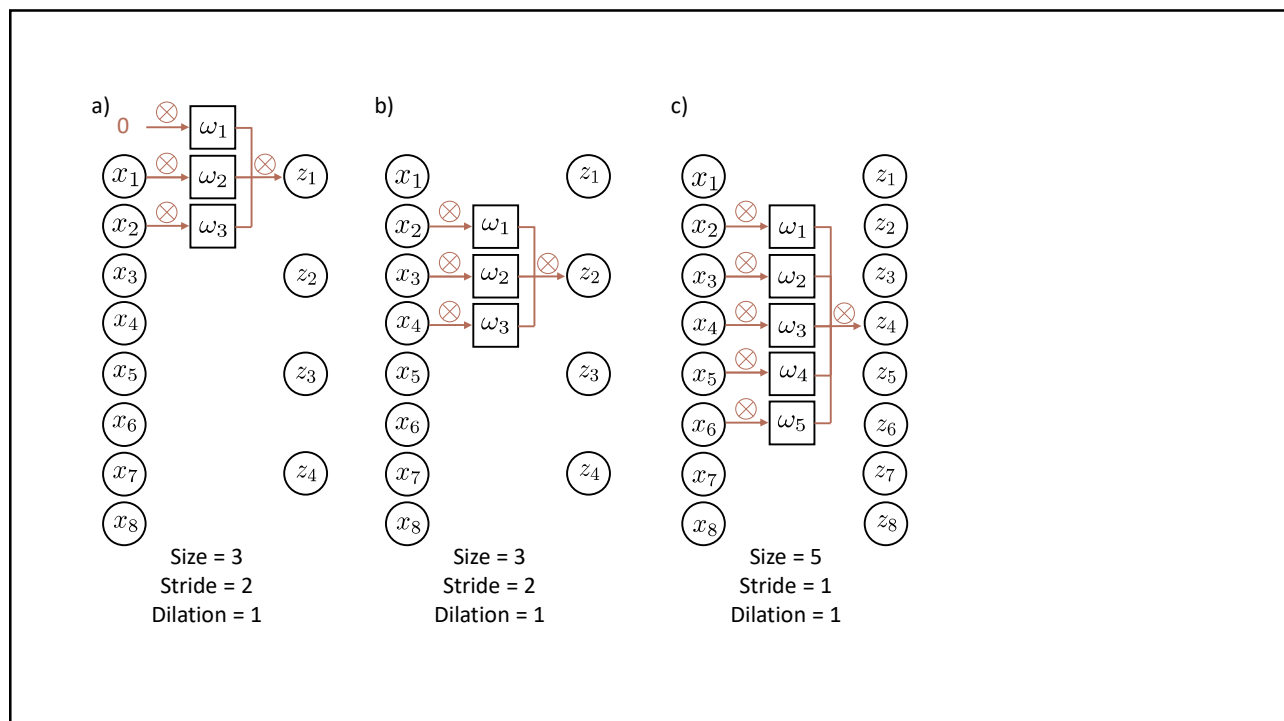
32



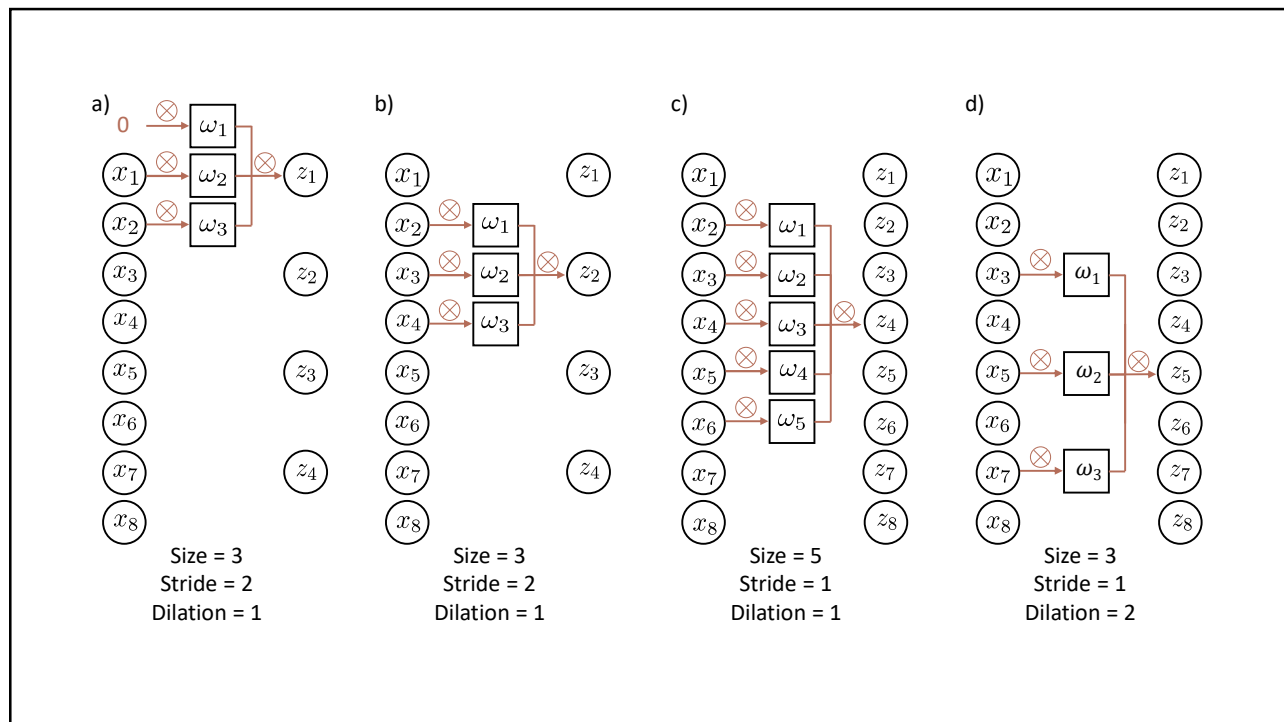
33



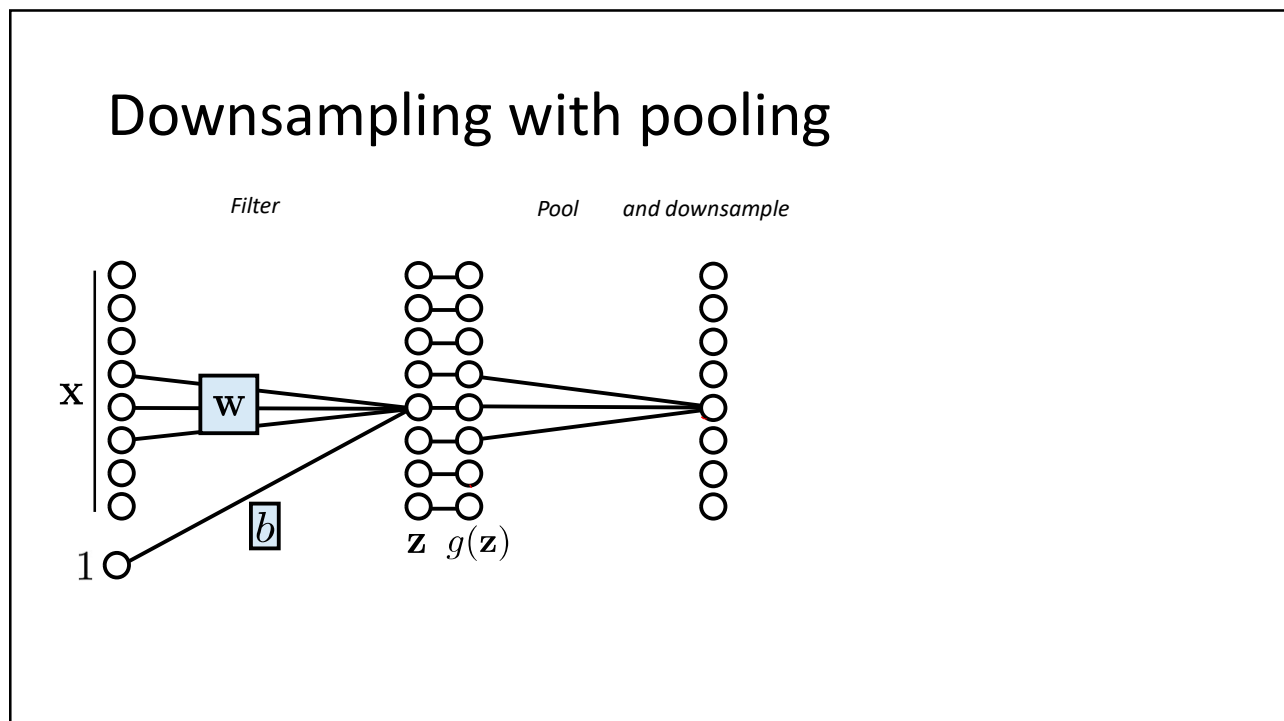
34



35

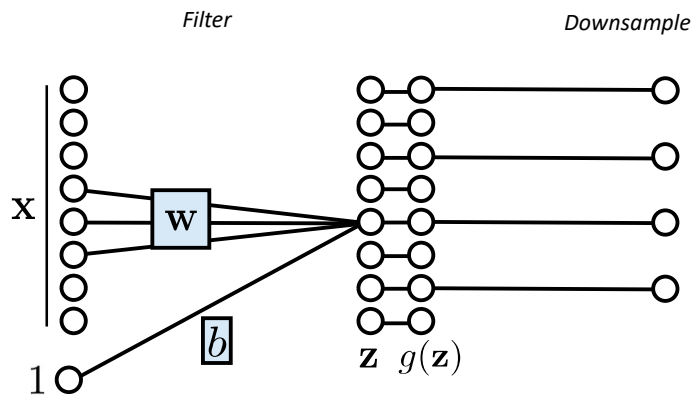


36



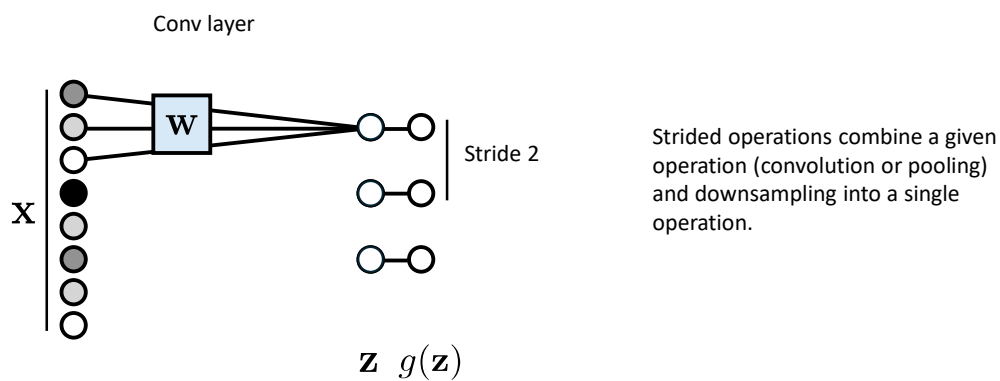
37

Downsampling



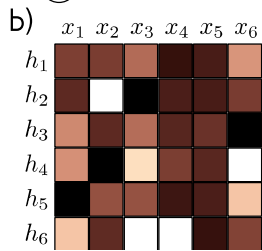
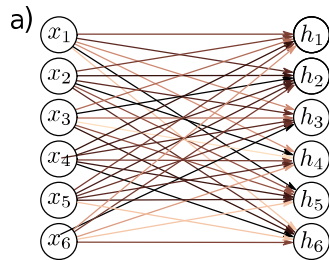
38

Strided operations



39

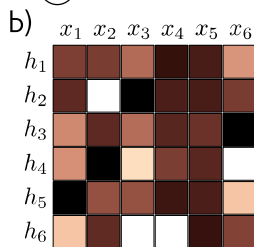
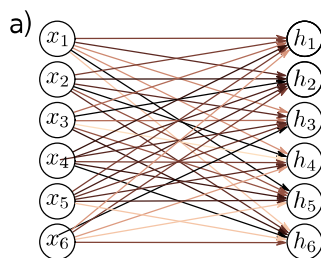
Special case of fully-connected network



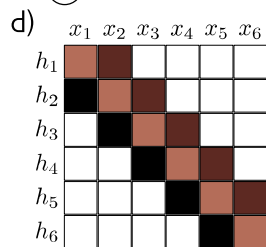
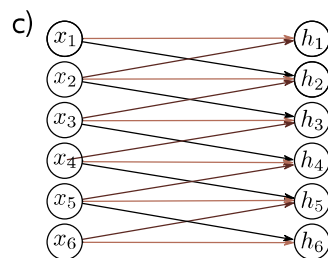
Fully connected network

40

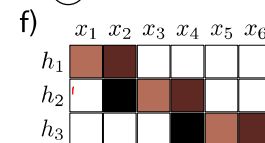
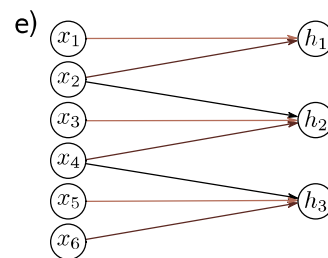
Special case of fully-connected network



Fully connected network



Convolution, size 3, stride 1, dilation 1, zero padding



Convolution, size 3, stride 2, dilation 1, zero padding

42

CNN Summary

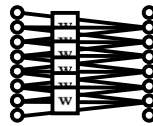
1. Equivariant with translation

$$f(\text{translate}(x)) = \text{translate}(f(x))$$

2. Patch processing

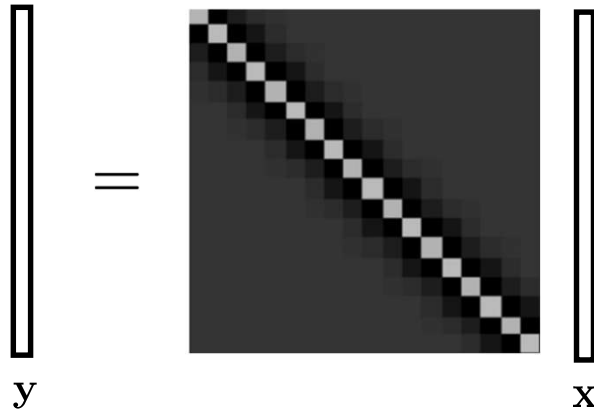
3. Image filter

4. Parameter sharing

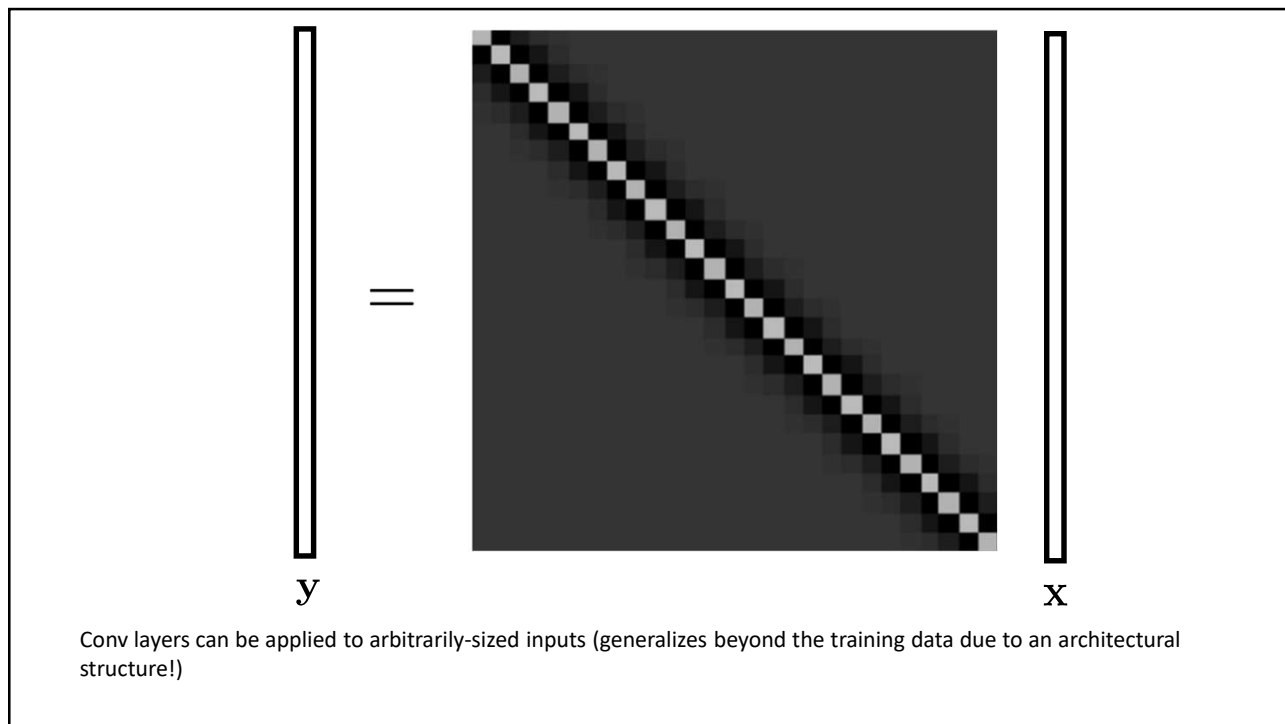


5. A way to process variable-sized tensors

43



44



45